

PROJECT REPORT

ER Editor

A Web-Based Chen-Notation Entity-Relationship Diagram Editor for SUPSI

Student	Sai Kiran Sabavath
Relatore	Brocco Amos
Committente	SUPSI DTI
Corso di laurea	MSE — Computer Science
Codice progetto	MP9_0001
Anno	2026
Data	

Contents

Abstract

Riassunto (Italiano)

Abstract (English)

Chapter 1 — Introduction

1.1 Context

1.2 Motivation

1.3 Objectives

1.4 Scope

1.5 Document Structure

Chapter 2 — Background & Requirements

2.1 Chen Notation

2.2 The Existing Tool and its XML Format

2.3 Moodle and the Integration Need

2.4 Functional Requirements

2.5 Non-Functional Requirements

Chapter 3 — Architecture & Design

3.1 Design Principles

3.2 Technology Stack

3.3 Layered Architecture

3.4 Data Model

3.5 State Management

3.6 Interaction Model

3.7 Notation Plugin System

Chapter 4 — Implementation

4.1 The Editing Canvas and Chen Tools

4.2 Validation Engine

4.3 File Format and Codecs

4.4 Moodle Integration

4.5 Responsive and Touch Interface

4.6 Documentation Site

Chapter 5 — Testing & Deployment

5.1 Test-Driven Development

5.2 The Test Suite

5.3 The Round-Trip Compatibility Gate

5.4 Continuous Integration

5.5 Deployment

Chapter 6 — Conclusions & Future Work

6.1 Results

6.2 Limitations

6.3 Future Work

6.4 Closing

Bibliography

Foundational

Platforms and standards

Libraries and tooling

Appendices

Appendix A — URL Parameters

Appendix B — Chen Validation Rules

Abstract

Riassunto (Italiano)

Lo studio dei modelli Entità-Relazione (ER) in notazione di Chen è una componente fondamentale dei corsi di basi di dati alla SUPSI. Lo strumento didattico in uso, *ERDesigner*, è un'applicazione desktop Java utilizzata da studenti e docenti. Questo progetto presenta una controparte basata su web che copre quanto offre *ERDesigner* e lo estende: riproduce la notazione di Chen, gira in qualsiasi browser moderno senza installazione, funziona su dispositivi touch e può essere incorporata nelle attività di Moodle per esercitazioni ed esami — mantenendo la compatibilità con i file di *ERDesigner*.

L'editor è costruito come una *single-page application* in React e TypeScript, con un'architettura a livelli (dominio → stato → interazione → canvas → interfaccia) che isola le regole del modello dalla loro rappresentazione visiva. La notazione di Chen è implementata come *plugin* — glyphi, regole di validazione e codec di importazione/esportazione sono moduli sostituibili — così che notazioni future (Crow's Foot, UML) possano essere aggiunte senza modificare il nucleo. Lo strumento mantiene la compatibilità di andata e ritorno con il formato XML dell'applicazione Java esistente, convalida i diagrammi rispetto alle regole della notazione di Chen e comunica con Moodle tramite l'API `postMessage` con salvataggio automatico.

Il risultato è uno strumento responsivo e multilingue, validato da quasi mille test automatici, distribuito pubblicamente e pronto per l'integrazione didattica. Il documento descrive il contesto, i requisiti, l'architettura, l'implementazione, la strategia di test e la distribuzione del progetto.

Abstract (English)

The study of Entity-Relationship (ER) models in Chen notation is a core component of the database courses at SUPSI. The established teaching tool, *ERDesigner*, is a Java desktop application used by students and instructors. This project presents a web-based counterpart that covers what *ERDesigner* does and extends it: it reproduces Chen notation, runs in any modern browser without installation, works on touch devices, and can be embedded inside Moodle activities for exercises and examinations — while remaining file-compatible with *ERDesigner*.

The editor is built as a single-page application in React and TypeScript, with a layered architecture (domain → state → interaction → canvas → user interface) that isolates the model rules from their visual representation. Chen notation is implemented as a plugin — glyphs, validation rules, and import/export codecs are swappable modules — so that future notations (Crow's Foot, UML) can be added without changing the core. The tool preserves round-trip compatibility with the XML format of the existing Java application, validates diagrams against Chen-notation rules, and communicates with Moodle through the `postMessage` API with automatic saving.

The result is a responsive, multilingual tool, validated by nearly one thousand automated tests, publicly deployed and ready for educational integration. This document describes the project's context, requirements, architecture, implementation, testing strategy, and deployment.

Chapter 1 — Introduction

1.1 Context

Entity-Relationship (ER) modelling is one of the foundational topics taught in the database courses at SUPSI (Scuola universitaria professionale della Svizzera italiana). Students learn to capture the structure of an application domain as a diagram of *entities*, *relationships*, and *attributes*, following the notation introduced by Peter Chen in 1976. Chen notation is favoured in teaching because it is explicit: every modelling concept — weak entities, identifying relationships, cardinality and participation constraints, key, multivalued, and composite attributes, and generalisations (IS-A hierarchies) — has a distinct visual symbol, which makes the underlying theory tangible for students.

To support this teaching, the department uses *ERDesigner*, a desktop application written in Java. ERDesigner is an established part of the courses: students draw ER diagrams with it and save them to an XML file that instructors can open and grade. It does its job well, and the workflow built around it is exactly what this project aims to preserve.

1.2 Motivation

ERDesigner works well as a desktop application. Moving the same capabilities into the browser, however, opens up possibilities the desktop form cannot offer on its own:

- **Zero installation.** A web tool runs in any browser, with nothing to download or keep up to date across the many machines and operating systems students use.
- **Touch and mobile.** Running in the browser brings the editor to the tablets and touch devices now common in classrooms, in addition to the desktop.
- **Moodle integration.** SUPSI delivers coursework and assessments through Moodle. A web editor can be embedded directly in a Moodle activity, so diagram exercises are assigned, completed, and collected in the same place as the rest of a course.

The project therefore builds a modern, web-based counterpart to ERDesigner: it covers what the desktop tool does and adds running anywhere, touch support, and Moodle integration — while staying fully interoperable with the diagrams instructors already have.

1.3 Objectives

The project set out to deliver a browser-based ER diagram editor that offers the capabilities above while remaining fully compatible with the existing teaching workflow. The concrete objectives were:

1. **Pure web application.** Run in any modern browser with no installation, as a single-page application.
2. **Chen-notation editing.** Provide an intuitive drag-and-drop canvas for creating and manipulating entities, relationships, and attributes — including composite attributes nested under a parent — as well as generalisations (IS-A hierarchies) in Chen notation, with undo/redo, multi-selection, zoom, and pan.
3. **Backward compatibility.** Import and export diagrams in the XML format of the existing Java application, with faithful round-trip fidelity so that the two tools can coexist.
4. **Diagram validation.** Check diagrams against the rules of Chen notation and surface errors to the student, with the ability to disable validation for exam scenarios.
5. **Moodle integration.** Allow instructors to embed the editor inside Moodle activities, with automatic saving of the student's work back to the host platform.
6. **Responsive and multilingual.** Adapt to desktop and tablet form factors, support touch interaction, and offer an English/Italian interface.

7. **Extensible and maintainable.** Adopt an architecture that keeps the model independent of its visual representation, so that additional notations can be added later without rewriting the core, and back the implementation with an automated test suite.

1.4 Scope

The work covers the editing vocabulary of classic ER modelling in Chen notation: entities (strong and weak), relationships (including identifying, recursive, and N-ary relationships, and the common 1:1 / 1:N / N:M cardinalities), attributes (key, discriminant, multivalued, derived, and composite), and generalisations (total and partial). Round-trip interoperability with the Java application's XML format and embedding inside Moodle are both in scope.

The following are explicitly **out of scope** for this iteration: a template or example library, multi-page diagrams, comments, real-time collaboration, and any form of cloud or server-side persistence. The editor remains a pure client-side application; persistence is delegated to the host (a downloaded file, or Moodle via `postMessage`). A second notation (such as Crow's Foot or UML) is not implemented, but the architecture is designed so that it could be added as a plugin without changing the core.

1.5 Document Structure

The remainder of this report is organised as follows:

- **Chapter 2 — Background & Requirements** introduces Chen notation, the existing ERDesigner tool and its file format, the Moodle integration need, and the functional and non-functional requirements.
- **Chapter 3 — Architecture & Design** describes the layered architecture, the technology stack, the notation-plugin system, and the data model.
- **Chapter 4 — Implementation** walks through the main features: the editing canvas, the validation engine, the Java XML codec, the Moodle bridge, the responsive interface, and the documentation site.
- **Chapter 5 — Testing & Deployment** covers the test-driven development approach, the automated test suite, and the public deployment including the Moodle host simulator.
- **Chapter 6 — Conclusions & Future Work** reviews what was achieved, the remaining limitations, and possible future directions.

Chapter 2 — Background & Requirements

2.1 Chen Notation

The Entity-Relationship model, introduced by Peter Chen in 1976, describes the structure of a domain using three primitives, each drawn with a distinct shape:

- **Entities** — the “things” of the domain (e.g. *Student*, *Course*), drawn as rectangles. A **weak entity**, which cannot be identified without another entity, is drawn with a double rectangle.
- **Relationships** — associations between entities (e.g. *enrols-in*), drawn as diamonds. A relationship is annotated with **cardinality** (1, N, M) and **participation** (partial or total) constraints on each connected entity. An **identifying relationship**, used to identify a weak entity, is drawn with a double diamond.
- **Attributes** — properties of entities or relationships (e.g. *name*, *date*), drawn as ellipses connected by a line. Attributes carry sub-types: a **key** attribute (underlined) identifies the entity; a **multivalued** attribute (double ellipse) can hold several values; a **derived** attribute (dashed ellipse) is computed from others; and a **composite** attribute groups child attributes beneath it.

In addition, **generalisation** (an IS-A hierarchy) expresses that several specialised entities are kinds of a more general one, drawn with a triangle and labelled total or partial. This explicit, symbol-rich vocabulary is precisely what makes Chen notation valuable for teaching, and reproducing it faithfully was a central requirement of the project.

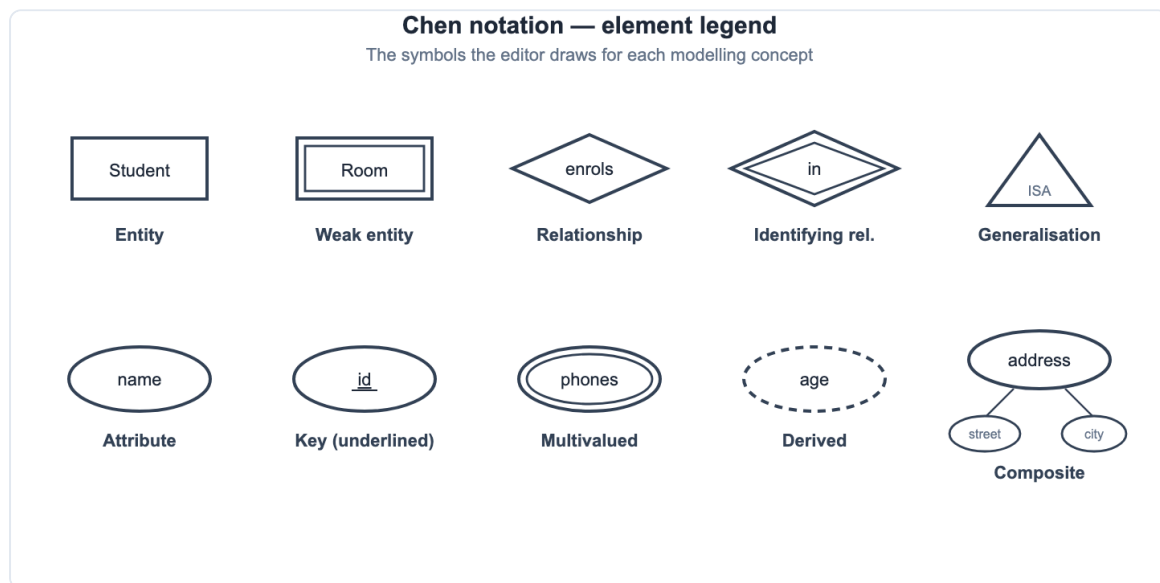


Figure 1: Chen notation element legend — the symbols the editor draws for entities, relationships, attribute variants, and generalisation.

2.2 The Existing Tool and its XML Format

The department’s existing tool, ERDesigner, is a Java desktop application that stores each diagram as an XML document. Two characteristics of this format shaped the project:

- **Center-based coordinates.** The Java application records the position of each shape by its centre point, whereas web rendering libraries conventionally use the top-left corner. Any interoperable web tool must convert between the two coordinate systems on load and save.
- **A specific element vocabulary.** The format encodes entities, relationships, attributes, and their Chen-notation sub-types (weak, identifying, key, multivalued, and so on) with particular element names and attributes.

Because instructors have an existing body of diagrams and an established grading workflow built around this format, **round-trip fidelity** — loading a file produced by the Java application, editing it, and saving it back so that the Java application can reopen it without loss — was treated as a hard acceptance criterion rather than a nice-to-have. The two tools must be able to coexist during any transition period.

2.3 Moodle and the Integration Need

Moodle is the open-source learning-management platform through which SUPSI delivers course materials and assessments. The pedagogical goal behind this project is for a student to complete a diagram exercise *inside* a Moodle activity — the same place they read the assignment and submit other work — rather than in a separate desktop application whose output must be uploaded by hand.

Technically, an embedded web tool runs inside an `<iframe>` on the Moodle page, and the two communicate through the browser's `postMessage` API, which allows scripts on different origins to exchange messages safely. This imposes several requirements on the editor:

- It must detect when it is running embedded and adapt its interface accordingly (the host owns saving, so the editor's own file menu is hidden).
- It must report the student's work back to the host automatically, so that progress is captured without the student remembering to "submit".
- It must support an **examination mode** in which validation feedback is suppressed, so that the tool does not coach students during a graded assessment.

These behaviours are driven by URL flags the host sets on the iframe (for example `embed`, `readonly`, `validation`, `examMode`, and `lang`), keeping the integration declarative and easy for an instructor to configure.

2.4 Functional Requirements

The functional requirements, derived from the project brief, are summarised below.

#	Requirement	Description
F1	Canvas editing	Create, move, resize, and connect entities, relationships, attributes, and generalisations on a zoomable, pannable canvas.
F2	Chen sub-types	Support weak entities, identifying/recursive/N-ary relationships, cardinality and participation, and key/multivalued/derived/composite attributes.
F3	Selection & batch ops	Single and marquee multi-selection, with move, copy/paste, and delete applied to the selection.
F4	Undo / redo	A full, reversible edit history.
F5	Validation	Check diagrams against Chen-notation rules and flag errors on the offending elements; the engine can be toggled off.
F6	Import / export	Load and save diagrams in the Java application's XML format with round-trip fidelity; additionally export a textual (Mermaid) representation.
F7	Moodle integration	Run embedded in an iframe, hide host-owned controls, auto-save via <code>postMessage</code> , and honour exam/read-only modes.
F8	Localisation	Switch the interface between English and Italian.
F9	Responsive & touch	Adapt the layout to desktop and tablet, and support touch gestures for creation, selection, and movement.

2.5 Non-Functional Requirements

#	Requirement	Target
N1	Zero installation	Runs in any modern browser as a single-page application; no plugins or runtime to install.
N2	Interaction performance	Smooth (≈ 60 fps) drag, pan, and zoom on diagrams of typical teaching size.
N3	Extensibility	Architecture allows a second notation (e.g. Crow's Foot, UML) to be added as a plugin without modifying the core model.
N4	Maintainability	Small, single-responsibility modules with enforced layer boundaries, so the system stays easy to navigate, test, and extend.
N5	Reliability	Backed by an automated test suite, with the Java XML round-trip as a hard regression gate.
N6	Accessibility	Keyboard-operable tools and shortcuts; readable contrast in light and dark themes.

Together these requirements frame the design decisions described in the next chapter: a layered architecture that isolates the model from its rendering, a plugin boundary for the notation, and a test strategy that protects the compatibility guarantees.

Chapter 3 — Architecture & Design

3.1 Design Principles

A diagram editor mixes three concerns that are easy to entangle: the model of *what a diagram is*, *how it is drawn*, and *how the user interacts with it*. Letting these bleed together produces large, hard-to-test files and makes new features expensive. The architecture is therefore organised around three principles:

- **Layered separation.** The application is split into layers with a strict, one-way dependency direction. The model of *what a diagram is* knows nothing about *how it is drawn* or *how the user interacts with it*.
- **Notation as a plugin.** Everything specific to Chen notation — the glyphs, the validation rules, the import/export formats — lives behind a single plugin interface, so the core never hard-codes notation details.
- **Small, single-responsibility modules.** Files are kept small (a soft target of around 200 lines), one component per file, with layer-boundary imports enforced by linting. This keeps each unit understandable and testable in isolation.

3.2 Technology Stack

Concern	Technology	Role
UI framework	React 19 + TypeScript	Component composition and type-safe model.
Build tooling	Vite 7	Fast dev server and optimised production bundle.
Diagram canvas	React Flow (xyflow) 12	Controlled node/edge rendering, pan/zoom, selection, with custom components per notation.
State management	Zustand 5	Small, sliced stores for diagram, viewport, selection, UI, and validation state.
Undo / redo	zundo	Temporal middleware wrapping the diagram store to give a history stack.
Interaction logic	XState 5	A finite-state machine governing drawing, placing, and connecting modes.
Styling	Tailwind CSS 4	Utility-first responsive styling and theming.
Internationalisation	i18next	English/Italian interface strings.
Testing	Vitest + Playwright	Unit/integration tests and end-to-end browser tests.

The whole application is a client-side single-page application; there is no server component.

3.3 Layered Architecture

The source is organised into layers, each a top-level folder under `src/`, with dependencies pointing only “downward” toward the model:

```
ui           → React components: toolbar, property panel, menus, modals
canvas       → React Flow integration and notation-adapter rendering
interaction  → the XState finite-state machine (drawing, placing, connecting)
state        → Zustand stores (diagram, viewport, selection, validation, ui)
notation     → the notation-plugin contract and the Chen implementation
domain       → pure model: types, geometry, graph helpers, invariants
```

Supporting these are `platform` (browser concerns: file access, i18n bootstrap) and `app` (composition root, URL-flag handling, and the Moodle bridge). The lowest layer, **domain**, is pure TypeScript with no framework dependencies: it defines what a diagram is and the operations on it. Each layer above adds one concern — state holding, then interaction, then rendering, then user-interface chrome — and may depend only on the layers below it. A lint rule rejects imports that cross these boundaries the wrong way, so the separation cannot quietly erode over time.

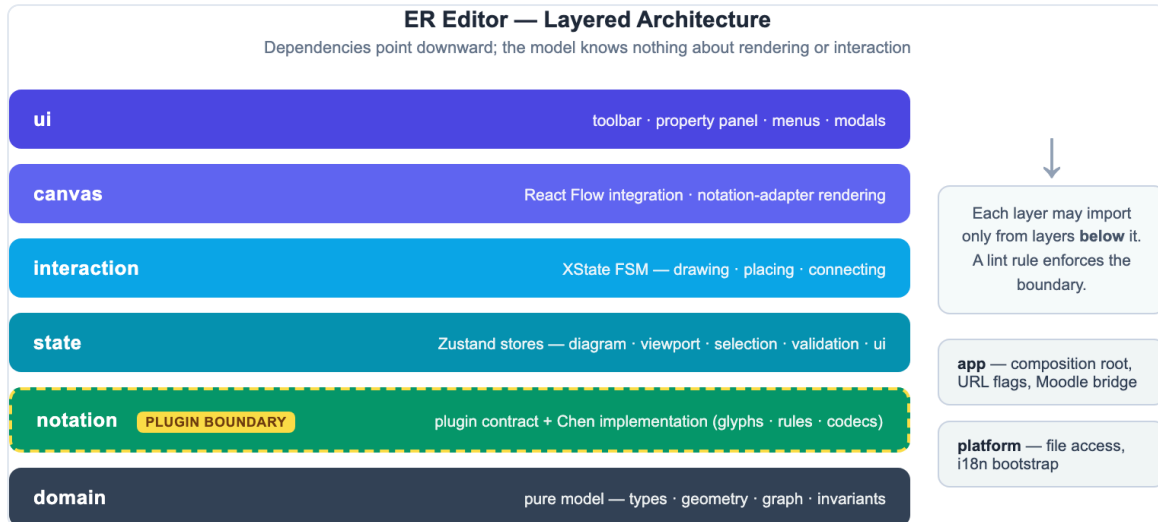


Figure 2: The layered architecture — dependencies point downward; the notation-plugin boundary is highlighted.

3.4 Data Model

A diagram is held as a single normalised, indexed structure rather than as nested object trees:

```
interface Diagram {
  schemaVersion: 1
  nodesById: Record<NodeId, ERNode>
  edgesById: Record<EdgeId, ERLink>
  nodeOrder: NodeId[]
  edgeOrder: EdgeId[]
}
```

Several deliberate choices follow from this:

- **Indexed by id.** Looking up or updating an element is a direct key access, and the `*Order` arrays preserve draw order. This scales better than searching nested arrays as diagrams grow.
- **Discriminated unions.** A node is one of `EntityNode`, `RelationshipNode`, `AttributeNode`, or `ISANode`, distinguished by a `kind` field; edges are likewise an `EntityRelationshipEdge`, `AttributeEdge`, or `ISAEEdge`. TypeScript then forces every piece of code that handles a node to account for each kind, which removes a large class of bugs at compile time.
- **Branded identifiers.** `NodeId` and `EdgeId` are branded string types that can only be produced through a dedicated factory, so an entity id can never be accidentally used where an edge id is expected.
- **Coordinate system.** Internally the editor uses top-left world coordinates, native to React Flow. The conversion to and from the Java application's centre-based coordinates is confined entirely to the Java XML codec (§4.3), so the rest of the system never has to reason about two coordinate conventions.

3.5 State Management

Application state is divided into several small Zustand stores, each owning one concern:

- **diagramStore** — the `Diagram` itself; the single source of truth for content.
- **viewportStore** — zoom level and pan offset.
- **selectionStore** — which nodes and edges are currently selected.
- **validationStore** — the current set of validation errors and whether validation is enabled.
- **uiStore** — interface state: open panels, modals, language, and the embed/readonly/exam flags.
- **interactionStore** — hosts the interaction state machine described next.

Splitting state this way means a component re-renders only when the slice it actually reads changes — for example, moving the viewport does not re-render the property panel. The diagram store is wrapped with the **zundo** temporal middleware, which records past and future states and so provides undo/redo essentially for free, scoped precisely to diagram content (and not, say, to which panel is open).

3.6 Interaction Model

User interaction with the canvas — picking a tool, placing a new element, dragging out a relationship, connecting two shapes — is inherently *modal*: the meaning of a click depends on what the user is currently doing. Rather than scatter this logic across event handlers with ad-hoc boolean flags, it is modelled explicitly as a **finite-state machine** using XState.

The machine has a small set of states — *selecting* (the default), *panning*, *placing* (a tool is armed and the next canvas click drops an element), *drawing*, and *quickRelationship* (dragging from one entity to another to create a relationship) — with well-defined transitions between them. Global events such as undo, redo, delete, duplicate, and rename are handled uniformly at the top level. Because the machine is a plain, framework-agnostic object, the interaction logic can be tested directly by sending it event sequences and asserting on the resulting state, with no DOM involved.

3.7 Notation Plugin System

The notation boundary is the architectural feature that keeps the tool extensible. A notation is described by a single `NotationPlugin` interface that bundles everything notation-specific:

```
interface NotationPlugin {
  id: string
  label: string
  nodeTypes: Record<NodeKind, ComponentType<NodeProps>> // glyph components
  edgeTypes: Record<EdgeKind, ComponentType<EdgeProps>> // connector components
  tools: ToolbarConfig // which tools appear
  defaults: NotationDefaults // default sizes, edge style
  validate: (d: Diagram) => Record<string, ValidationError[]>
  codecs: { nativeJson: Codec; javaXml?: Codec; mermaid?: Codec; png?: Codec; svg?: Codec }
}
```

Chen notation is supplied as one implementation of this interface: it provides the rectangle/diamond/ellipse/triangle glyph components, the cardinality and participation rendering, its validation rule set (§4.2), and its codecs (§4.3). The core canvas and UI consume only the interface — they ask the active plugin for the component that draws a given node kind, or for the rules to validate against — and never reference Chen specifically. Adding a second notation such as Crow's Foot would mean writing a new plugin object; the core model, stores, and interaction machine would not change. This directly satisfies the extensibility requirement (N3) set out in the previous chapter.

Chapter 4 — Implementation

This chapter describes how the main features were built on top of the architecture of Chapter 3. It follows the path of a diagram through the system: drawing it on the canvas, validating it, saving and exporting it, and — when embedded — exchanging it with Moodle. It closes with the responsive interface and the project’s documentation site.

4.1 The Editing Canvas and Chen Tools

The canvas is built on React Flow, which provides pan, zoom, selection, and node/edge rendering, while the Chen-specific glyphs are supplied by the notation plugin. The toolbar exposes the modelling vocabulary in three groups:

- **Selection** — *select* and *pan*.
- **Elements** — *entity*, *relationship*, and *attribute*, placed by dragging onto the canvas or by arming the tool and clicking.
- **Connections** — a plain *connect* tool plus *quick* flows that create a relationship with a preset cardinality in one gesture (1:1, 1:N, N:M) and two generalisation flows (partial and total IS-A).

A deliberate design detail concerns generalisations: a lone IS-A node has no meaning in Chen notation, since it must join a parent to one or more children. The toolbar therefore does not offer a standalone “ISA” element; instead the two quick-generalisation flows create the triangle together with its connections in a single action. Each created element is editable through a property panel that adapts to the selected kind — toggling an entity’s *weak* flag, an attribute’s *key/multivalued/derived/composite* flags, or a relationship participant’s cardinality and participation.

All structural edits go through the diagram store, which is wrapped with undo/redo history, so every action — create, move, rename, delete, or a multi-element batch operation — is individually reversible.

4.2 Validation Engine

The validation engine checks a diagram against the rules of Chen notation and reports problems on the offending element. Rules are small, independent functions, each taking the whole diagram and returning the elements that violate it, grouped into five categories:

Category	Examples
Entity	A strong entity must have a key; a weak entity must have a discriminant.
Relationship	Cardinality/participation must be specified; identifying relationships must connect a weak entity.
Attribute	A key and a discriminant are mutually exclusive; names must be present.
Generalisation	An IS-A must have a parent and at least one child.
Structural	Names must be unique where the notation requires it.

In total the engine ships **more than thirty rules**. Crucially, rules encode the *semantics* of the notation, not just surface checks: for instance, the “entity must have a key” and “entity must have an attribute” rules deliberately exempt IS-A children, because a subclass inherits its identifier from its parent — flagging it would teach the student the wrong thing.

Errors surface in two places: a small marker on the element in the canvas, and a list in the property panel describing each problem. The whole engine can be switched off through configuration, which is the mechanism behind the examination mode discussed in §4.4.

4.3 File Format and Codecs

Persistence and interchange are handled by *codecs* — modules that translate between the internal diagram model and an external text format. The notation plugin can carry several; the Chen plugin provides four paths exposed through the application menu:

- **Java XML (Open / Save)** — the primary format, fully round-trippable with the ERDesigner desktop application. This codec is where the coordinate-system conversion lives: ERDesigner stores shape positions by their centre point, so on *load* the codec converts centre coordinates to the top-left convention the editor uses internally, and on *save* it converts back. Round-trip fidelity — open a file produced by the desktop tool, edit it, save it, and have the desktop tool reopen it without loss — is verified by automated tests against real SUPSI sample files (§5).
- **Mermaid (Export)** — a textual representation of the diagram, useful for embedding in documentation or Markdown-based materials.
- **PNG and SVG (Export)** — raster and vector images of the canvas, for inclusion in reports or submissions.

Because each format is an independent codec behind a common interface, adding or changing a format does not touch the editing or rendering code.

4.4 Moodle Integration

The Moodle integration lets a student work on a diagram inside a Moodle activity, with their work captured automatically by the host page. It is implemented as a small *bridge* that mediates between the editor and the surrounding page through the `postMessage` API.

Activation. The bridge only activates when the editor is genuinely embedded — that is, when it is launched with the `embed` flag *and* is actually running inside a parent frame. Run directly, the editor behaves as a normal standalone application and the bridge stays dormant.

Message protocol. Communication is a small set of typed messages, each tagged with a `source` field so the two sides can ignore unrelated traffic on the page:

Direction	Message	Meaning
Editor → host	<code>ready</code>	The editor has loaded and is ready to receive a diagram.
Host → editor	<code>init / load</code>	Load this XML diagram into the editor.
Editor → host	<code>autosave</code>	The diagram changed; here is its current XML (debounced).
Editor → host	<code>save</code>	A final snapshot, sent when the page is about to unload.
Editor → host	<code>error</code>	Parsing or serialisation failed; here is the reason.

Automatic saving. Rather than send a message on every keystroke, autosave is *debounced*: after a change, the bridge waits 800 ms for activity to settle before serialising the diagram and posting it to the host. This coalesces a burst of edits into a single message. To avoid losing the last edits, a final `save` is flushed when the page is hidden or unloaded.

Security. Cross-origin messaging must be told which origin to trust. The bridge resolves the target origin in order of preference: an explicit `parentOrigin` URL parameter set by the host, then the document referrer, falling back to a wildcard only as a last resort (and logging a warning when it does). Incoming messages whose origin does not match are ignored.

Exam mode. When the host requests examination mode, validation feedback is suppressed and the file controls are hidden, so the tool records the student's work without coaching them during a graded assessment. All of these behaviours — embed, read-only, validation on/off, exam mode, and interface language — are driven

declaratively by URL flags the host sets on the iframe, so an instructor configures the integration without writing code.

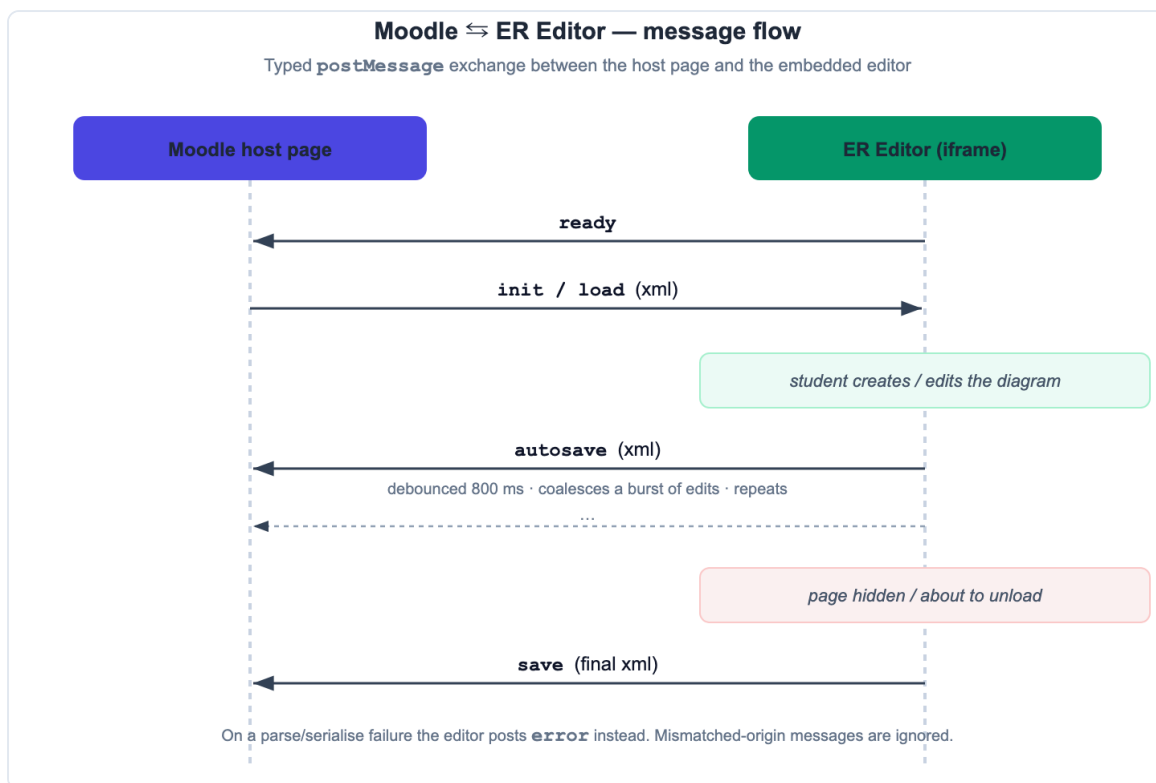


Figure 3: The Moodle ⇌ editor `postMessage` exchange: `ready` → `init/load` → `debounced autosave` → `final save` on unload.

4.5 Responsive and Touch Interface

The interface adapts to three layout modes — *desktop*, *tablet*, and *mobile* — detected from the viewport using media queries. The property panel renders inline beside the canvas on desktop, and as a slide-over drawer on smaller screens. The toolbar becomes a vertical rail that can be collapsed to a single button on phones, so it never covers the limited canvas area; picking a tool auto-collapses it. Touch gestures drive creation, selection, and movement, so the editor is usable on the tablets common in classrooms, extending ER modelling to devices beyond the desktop.

4.6 Documentation Site

Alongside the application, the project ships a documentation site built with VitePress. It serves two audiences from one source: a high-level **user guide** (tools, keyboard shortcuts, file handling, and how to embed the editor in Moodle) and a detailed **developer guide** (architecture, the domain model, the interaction machine, the notation plugin contract, and recipes for extending the tool). Selected pages embed code directly from the source via region markers, and an API reference is generated automatically from the TypeScript types, so the documentation stays in step with the code rather than drifting from it. The site is deployed together with the application (§5.3).

Chapter 5 — Testing & Deployment

Reliability was a stated non-functional requirement (N5), and for a tool that must interoperate with an existing application it is not optional: a diagram that fails to round-trip is a corrupted piece of a student's work. This chapter describes the testing approach that protects those guarantees and the deployment that delivers the tool.

5.1 Test-Driven Development

Features were built test-first: a failing test was written to capture the intended behaviour, the minimal code to satisfy it followed, and the design was refined once the test passed. This discipline was practical here precisely because of the architectural choices in Chapter 3. The pure domain layer, the framework-agnostic interaction machine, and the rule functions are all plain inputs-to-outputs code with no DOM or network dependencies, so they can be tested directly and quickly. Writing the test first also forced each unit to have a usable interface before its internals existed, which kept the modules small and focused.

5.2 The Test Suite

The suite uses **Vitest** for unit and integration tests and **Playwright** for end-to-end browser tests. In total there are roughly **950 tests across about 130 test files**, distributed across every layer of the architecture rather than concentrated in one place:

- **Domain** — geometry, graph helpers, and model invariants.
- **Notation** — each Chen validation rule, and the codecs (read, write, transform, and round-trip).
- **State & interaction** — store reducers, selectors, and the interaction machine driven by event sequences.
- **UI** — components and overlays (the property panel, toolbar, menus, inline rename) rendered and asserted with the React Testing Library.

Time-dependent behaviour is tested deterministically with simulated timers — the 800 ms autosave debounce of §4.4, for example, is verified by advancing virtual time rather than waiting in real time. End-to-end coverage is provided by a Playwright smoke test that drives a real browser to confirm the application boots and the core flow works.

5.3 The Round-Trip Compatibility Gate

The single most important guarantee is interoperability with the ERDesigner XML format, so it has dedicated, layered tests. The Java XML codec is tested at the level of reading, writing, and coordinate transformation individually, and then end-to-end: real SUPSI sample files (for example a conference-schema diagram) are loaded, re-serialised, and checked to ensure the output is faithful to the input. Because the centre-versus-top-left coordinate conversion (§4.3) is a common source of drift, these round-trip tests act as a regression gate — a change that subtly shifts a coordinate or drops an attribute fails the build rather than silently corrupting files.

5.4 Continuous Integration

Every change runs through an automated pipeline before it can be merged. The main workflow performs, in order:

1. **Lint** — code style and the layer-boundary import rules.
2. **Locale lint** — a check that the English and Italian translation bundles are complete and consistent.
3. **Type check** — the full TypeScript compilation.
4. **Unit & integration tests** — the Vitest suite.
5. **Build** — a complete production build, which must succeed.

A separate workflow runs the Playwright end-to-end test in a real browser. Because the build step is part of the gate, a broken build — including a broken documentation build — cannot reach the deployed site.

5.5 Deployment

The application is deployed on **Vercel** as static assets, with no server runtime. A single build command produces one `dist/` directory that contains everything served:

- the **editor** at the site root (`/`);
- the **documentation site** under `/docs/` ;
- the **Moodle host simulator** at `/moodle` .

The build compiles the application, builds the documentation site, and copies the documentation output into the editor's `dist/docs` folder, so the two ship together from a single deployment. Routing is configured so that clean URLs work and any unknown path falls back to the single-page application, while the documentation, the simulator, and static assets are served directly.

The Moodle host simulator deserves a note: it is a small page that embeds the editor in an iframe exactly as a real Moodle activity would, and displays the `postMessage` traffic live. It made the integration of §4.4 testable during development without a running Moodle instance, and it doubles as runnable documentation of the host side of the protocol.

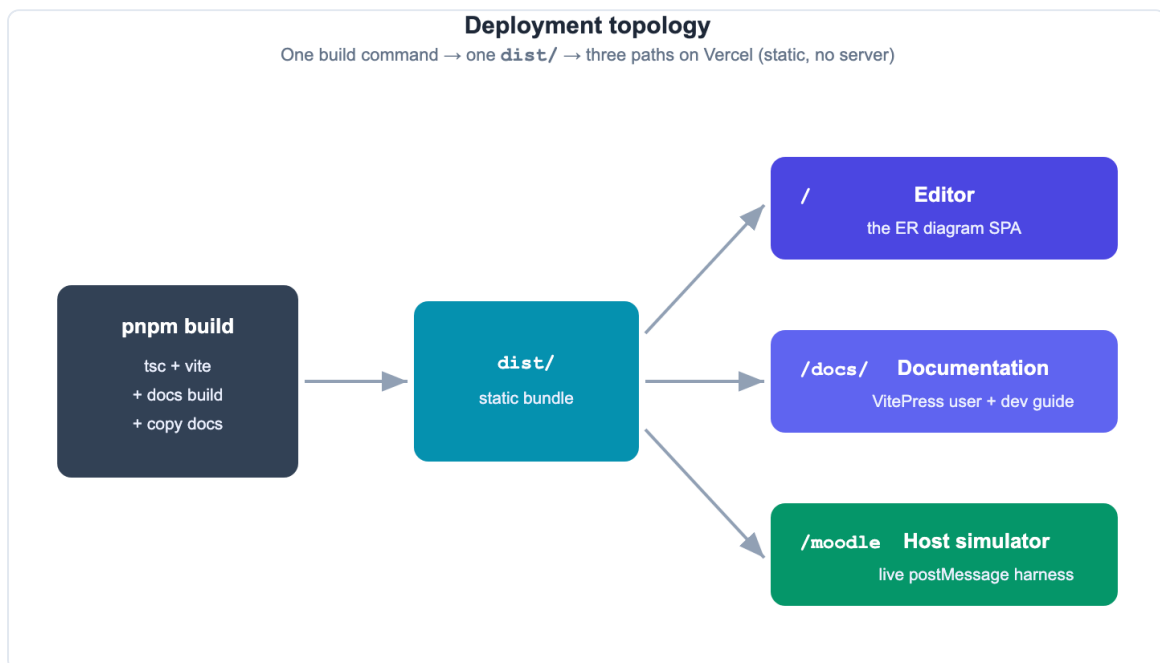


Figure 4: Deployment topology — one build produces a single `dist/` served at three paths: `/` (editor), `/docs/` (documentation), `/moodle` (simulator).

Chapter 6 — Conclusions & Future Work

6.1 Results

The project delivered a working, deployed ER diagram editor that meets the objectives set out in Chapter 1. Measured against those objectives:

#	Objective	Outcome
1	Pure web application	A single-page application running in any modern browser with no installation.
2	Chen-notation editing	Entities, relationships, attributes (key, multivalued, derived, composite), and generalisations, with undo/redo, multi-selection, zoom, and pan.
3	Backward compatibility	Round-trip Open/Save in the ERDesigner XML format, verified against real SUPSI sample files.
4	Diagram validation	More than thirty Chen-notation rules, surfaced on the canvas and in the panel, and toggleable for exams.
5	Moodle integration	An embeddable mode with debounced autosave over <code>postMessage</code> , plus a host simulator for testing.
6	Responsive & multilingual	Desktop, tablet, and mobile layouts with touch support, and an English/Italian interface.
7	Extensible & maintainable	A layered architecture with a notation-plugin boundary, backed by roughly 950 automated tests.

Beyond the editor itself, the project produced a documentation site serving both users and developers, and a combined deployment that ships the editor, the documentation, and the Moodle simulator from a single build. The result is a tool ready for use in teaching: it brings ER modelling to the browser, to touch devices, and into Moodle, while remaining fully interoperable with the diagrams instructors already use.

6.2 Limitations

Several boundaries are worth stating honestly:

- **One notation in practice.** The architecture is designed so that a second notation can be added as a plugin, and Chen notation is implemented entirely through that boundary. However, only Chen is implemented, so the extensibility of the seam has not yet been proven by a second, independent notation.
- **Light end-to-end coverage.** Unit and integration coverage is extensive, but automated browser-level (end-to-end) testing currently amounts to a single smoke test. Much of the interaction behaviour is exercised through the interaction machine in isolation rather than through a real browser.
- **Deliberately excluded features.** Real-time collaboration, multi-page diagrams, a template/example library, comments, and any cloud or server-side persistence were out of scope by design. Persistence is delegated to the host — a downloaded file, or Moodle.
- **Single interchange format.** The only fully round-trippable format is the ERDesigner XML; Mermaid, PNG, and SVG are export-only.

6.3 Future Work

The most natural extensions follow directly from the design:

- **A second notation.** Implementing Crow's Foot or UML notation as a plugin would both add a frequently requested capability and validate the extensibility claim of the architecture.
- **Broader end-to-end testing.** Expanding the Playwright suite — and adding visual regression tests for the glyphs — would protect the interaction and rendering layers as directly as the round-trip tests protect the file format.
- **Auto-layout.** Automatic arrangement of entities and relationships would help students tidy diagrams and ease the import of large legacy files.
- **Accessibility audit.** A formal pass on keyboard navigation, focus order, and screen- reader labelling would broaden the tool's reach.
- **Selective reintroduction of deferred features.** Should teaching needs demand it, the excluded features — for example a small library of example schemas — could be added incrementally on top of the existing architecture.

6.4 Closing

The project set out to bring the capabilities of the established ERDesigner tool to the browser — a modern, web-based, Moodle-ready ER editor that stays compatible with the existing teaching workflow. The editor is deployed, interoperable, validated by an extensive test suite, and documented for both its users and its future maintainers. Its layered, plugin-oriented architecture leaves it well positioned to grow — toward new notations, richer testing, and whatever the courses at SUPSI require next.

Bibliography

Foundational

- Chen, P. P. (1976). *The Entity-Relationship Model — Toward a Unified View of Data*. ACM Transactions on Database Systems, 1(1), 9–36.

Platforms and standards

- Moodle — open-source learning management system. <https://moodle.org>
- MDN Web Docs — *Window.postMessage() method*. <https://developer.mozilla.org/en-US/docs/Web/API/Window/postMessage>

Libraries and tooling

- React. <https://react.dev>
- TypeScript. <https://www.typescriptlang.org>
- Vite. <https://vite.dev>
- React Flow (xyflow) — node-based UI library. <https://reactflow.dev>
- Zustand — state management for React. <https://github.com/pmndrs/zustand>
- XState — state machines and statecharts for JavaScript/TypeScript. <https://stately.ai/docs/xstate>
- Tailwind CSS. <https://tailwindcss.com>
- i18next — internationalisation framework. <https://www.i18next.com>
- Vitest — unit test framework. <https://vitest.dev>
- Playwright — end-to-end browser testing. <https://playwright.dev>
- VitePress — static site generator for technical documentation. <https://vitepress.dev>

Note: the references above point to the official documentation of each technology used in the project. Final formatting (citation style, access dates) should be aligned with the SUPSI submission guidelines before hand-in.

Appendices

Appendix A — URL Parameters

The editor reads a small set of URL flags at start-up. A Moodle host configures the integration declaratively by setting these on the embedded iframe's URL; they can also be used directly for testing.

Flag	Values	Effect
<code>lang</code>	<code>en</code> <code>it</code>	Sets the interface language.
<code>validation</code>	<code>on</code> <code>off</code>	Enables or disables the validation engine.
<code>readonly</code>	<code>present</code> / <code>absent</code>	Restricts the tools to selection and pan; mutations are blocked.
<code>embed</code>	<code>present</code> / <code>absent</code>	Embed mode: hides the file menu (the host owns Save/Open/Export) and activates the Moodle bridge. The drawing toolbar stays visible so the student can still build the diagram.
<code>examMode</code>	<code>present</code> / <code>absent</code>	Forces validation off and disables the Save/Export/validation controls, so the tool does not coach the student during a graded assessment.
<code>parentOrigin</code>	an origin URL	The origin the editor trusts as the message target for <code>postMessage</code> ; used to address autosave/save messages to the host.

Appendix B — Chen Validation Rules

The validation engine ships 33 rules, grouped by category below. Each is an independent check that takes the whole diagram and returns the elements that violate it. Rules with severity *error* mark the diagram invalid; *warning* rules flag likely mistakes without blocking. Messages are shown in their English form; placeholders such as “...” and *n* are filled in at runtime.

Entity (10)

Severity	Rule	Meaning
error	<code>must-have-key</code>	Strong entity must have at least one key attribute.
error	<code>weak-missing-discriminant</code>	Weak entity must have a discriminant attribute.
error	<code>must-have-attribute</code>	Entity must have at least one attribute.
error	<code>weak-total-participation</code>	Weak entity must have total participation in the identifying relationship.
error	<code>weak-not-on-1-side</code>	Weak entity cannot be on the 1-side of the identifying relationship (must be N-side).
error	<code>weak-single-identifying</code>	Weak entity must connect to exactly one identifying relationship.
error	<code>name-unique</code>	Entity name is already used by another entity.
error	<code>name-non-empty</code>	Entity name must not be empty.
warning	<code>orphan-warning</code>	Entity has no relationships (may be incomplete).

Severity	Rule	Meaning
error	weak-no-key	Weak entity must not have a key attribute; use a discriminant (partial key) instead.

Relationship (7)

Severity	Rule	Meaning
error	min-two-entities	Relationship must connect at least 2 entities (or be a recursive relationship).
error	cardinality-required	All connections on this relationship must have cardinality defined.
error	participation-required	All connections on this relationship must have participation defined.
error	identifying-needs-weak	Identifying relationship must connect at least one weak entity.
error	non-identifying-not-weak	Non-identifying relationship is connected to a weak entity; it should probably be marked identifying.
error	name-unique	Relationship name is already used by another relationship.
error	recursive-roles-distinct	Recursive relationship must have distinct, non-empty roles on both edges.

Attribute (12)

Severity	Rule	Meaning
error	single-parent	Attribute must connect to exactly one parent (entity, relationship, or composite attribute).
error	no-dual-parent	Attribute cannot connect to both an entity and a relationship.
error	discriminant-weak-only	Discriminant is only valid on attributes of a weak entity.
error	not-key-and-derived	Attribute cannot be both key and derived.
error	key-not-multivalued	Key attribute cannot be multivalued.
error	discriminant-not-multivalued	Discriminant cannot be multivalued.
error	relationship-not-key	Attributes of a relationship cannot be key attributes.
error	name-unique	Attribute name is already used by another attribute on the same parent.
error	composite-needs-sub	Composite attribute should have at least one sub-attribute.
error	sub-not-key	Sub-attribute cannot be a key attribute.
error	sub-not-discriminant	Sub-attribute cannot be a discriminant.
error	sub-not-composite	Sub-attribute cannot itself be composite (only one level of nesting).

Generalisation (3)

Severity	Rule	Meaning
error	parent-exists	A generalisation (IS-A) must have a parent entity.
error	has-children	A generalisation (IS-A) must have at least one child.
warning	single-child-warning	An IS-A with a single child is semantically weak — consider whether specialisation is meaningful.

Structural (1)

Severity	Rule	Meaning
error	dangling-edge	An edge references a node that does not exist.